comparable and comparator

JAVA STUDY GUIDE & CODE EXAMPLES

Comparable and Comparator in Java

Both are interfaces used to define how objects of a class are ordered.

It helps to write our own logic for comparing objects.

They are what TreeSet, TreeMap, Collections.sort(), Arrays.sort(), and PriorityQueue rely on when they need to sort or compare objects.

The essential difference:

Comparable: the class itself defines its one natural ordering.

Comparator: an external object defines any number of alternative orderings.

Comparable<T>

Lives in java.lang. One method: int compareTo(T other).

The class itself implements it, so the ordering is built into the class.

There is exactly one natural order per class (e.g. Integer by numeric value, String lexicographically).

Used automatically by Collections.sort(list), Arrays.sort(arr), TreeSet, TreeMap, PriorityQueue when no comparator is supplied.

Method: int compareTo(T other);

Return value contract (for a.compareTo(b)):

Example: Student Class sorted based on the age

```
import java.util.ArrayList; import java.util.Collections; import java.util.Comparator; import
java.util.List; class Students implements Comparable<Students> { int age; String name; public
Students(int age, String name) { this.age = age; this.name = name; } @Override public int
compareTo(Students o) { if (this.age > o.age) return 1; else return -1; } @Override public String
toString() { return "Student Age=" + age + ", Name=" + name; } } public class ComparableDemo { public
static void main(String[] args) { List<Students> students = new ArrayList<Students>();
students.add(new Students(23, "Anitha")); students.add(new Students(21, "Rahul")); students.add(new
Students(22, "Kiran")); students.add(new Students(25, "Navin")); Collections.sort(students); for
(Students s : students) System.out.println(s); }
```

}

2. Comparator

Lives in java.util.

Main method: int compare(T a, T b).

Implemented in a separate class, an anonymous class, or (most commonly now) a lambda or method reference.

You can define many orderings for the same class without modifying it, which also means it works for classes you can't edit (library classes).

Passed explicitly: Collections.sort(list, cmp), list.sort(cmp), new TreeSet<>(cmp), new PriorityQueue<>(cmp).

Example Program for sorting ArrayList of Strings based on their length.

```
import java.util.*; public class CompareDemo { public static void main(String[] args) {
Comparator<String> com=new Comparator<String>() { @Override public int compare(String s1,String s2)
{ if (s1.length()>s2.length()) return 1; else return -1; } }; List<String> s=new ArrayList<>();
```

```
s.add("Anitha"); s.add("Navin"); s.add("John"); s.add("Jo"); Collections.sort(s,com);
System.out.println(s); } }
```

Output:

[Jo, John, Navin, Anitha]

Example: Student Class sorted based on the age (Here the comparator is written outside the student class

```
import java.util.ArrayList; import java.util.Collections; import java.util.Comparator; import
java.util.List; class Student { int age; String name; public Student(int age, String name) { this.age
= age; this.name = name; } @Override public String toString() { return "Student Age=" + age + ",
Name=" + name; } } public class StudentClassCompare { public static void main(String[] args) {
Comparator<Student> com = new Comparator<Student>() { public int compare(Student s1, Student s2) { if
(s1.age > s2.age) return 1; else return -1; } }; List<Student> students = new ArrayList<Student>();
students.add(new Student(23, "Anitha")); students.add(new Student(21, "Rahul")); students.add(new
Student(22, "Kiran")); students.add(new Student(25, "Navin")); Collections.sort(students,com); for
(Student s : students) System.out.println(s); } }
```

Return	Meaning
negative	a comes before b
zero	a and b are equal in ordering
positive	a comes after b